

НИУ ИТМО
Факультет ПИиКТ

Лабораторная работа №4

Аппроксимация функции методом наименьших квадратов

Вариант №5

Преподаватель **Наумова Надежда Александровна**
Подготовил **Лоскутов Прохор Александрович**

Санкт - Петербург 2026

1 из 26

Оглавление

Оглавление	2
1 Вычислительная реализация задачи	3
1.1 Таблица табулирования заданной функции	3
1.2 Линейная аппроксимация	3
1.3 Квадратичная аппроксимация	5
1.4 Сравнение и выбор наилучшего приближения	6
2 Программная реализация задачи	6
2.1 Рабочие формулы и блок-схемы реализованных моделей .	7
2.2 Вспомогательные функции, общие для всех методов	11
2.3 Сообщение по коэффициенту детерминации R^2	17
2.4 Исходный код основных элементов расчёта	17
2.5 Интерфейс приложения	25
3 Выводы	25

1 Вычислительная реализация задачи

По варианту №5 необходимо построить аппроксимации заданной табличной функции методом наименьших квадратов:

$$y = \frac{6x}{x^4 + 5}, \quad x \in [0, 2], \quad h = 0.2$$

1.1 Таблица табулирования заданной функции

Таблица 1

Значения функции $y = 6x/(x^4 + 5)$ в 11 узлах

i	x_i	y_i
1	0.0	0.000000
2	0.2	0.239923
3	0.4	0.477555
4	0.6	0.701809
5	0.8	0.887311
6	1.0	1.000000
7	1.2	1.017869
8	1.4	0.950054
9	1.6	0.830910
10	1.8	0.696882
11	2.0	0.571429

1.2 Линейная аппроксимация

В качестве эмпирической формулы выбираем линейную функцию $\varphi(x) = ax + b$. Метод наименьших квадратов сводится к минимизации суммы квадратов отклонений:

$$S(a, b) = \sum_{i=1}^n (ax_i + b - y_i)^2 \rightarrow \min$$

Приравнивая частные производные $\partial S/\partial a$ и $\partial S/\partial b$ к нулю, получаем систему нормальных уравнений (см. лекцию):

$$\begin{cases} a \cdot S_{XX} + b \cdot S_X = S_{XY} \\ a \cdot S_X + b \cdot n = S_Y \end{cases}$$

где введены обозначения:

$$S_X = \sum_{i=1}^n x_i, \quad S_{XX} = \sum_{i=1}^n x_i^2, \quad S_Y = \sum_{i=1}^n y_i, \quad S_{XY} = \sum_{i=1}^n x_i y_i$$

Вычисляем суммы по таблице 1:

$$S_X = 11.000000, \quad S_{XX} = 15.400000,$$

$$S_Y = 7.373743, \quad S_{XY} = 8.648161$$

По правилу Крамера для системы 2×2 :

$$\begin{aligned} \Delta &= S_{XX} \cdot n - S_X \cdot S_X \\ &= 15.4 \cdot 11 - 11 \cdot 11 = 169.4 - 121 = 48.4 \end{aligned}$$

$$\begin{aligned} \Delta_1 &= S_{XY} \cdot n - S_X \cdot S_Y \\ &= 8.648161 \cdot 11 - 11 \cdot 7.373743 \\ &= 95.129768 - 81.111169 = 14.018601 \end{aligned}$$

$$\begin{aligned} \Delta_2 &= S_{XX} \cdot S_Y - S_X \cdot S_{XY} \\ &= 15.4 \cdot 7.373743 - 11 \cdot 8.648161 \\ &= 113.555646 - 95.129768 = 18.425867 \end{aligned}$$

$$a = \Delta_1 / \Delta = 14.018601 / 48.4 = 0.289641$$

$$b = \Delta_2 / \Delta = 18.425867 / 48.4 = 0.380700$$

Итоговая модель: $\varphi_1(x) = 0.289641x + 0.380700$.

Проверим выбор модели — вычислим $\varphi(x_i)$ и отклонения $\varepsilon_i = \varphi(x_i) - y_i$:

Таблица 2

Линейная аппроксимация: значения и отклонения

i	x_i	$\varphi_1(x_i)$	ε_i
1	0.0	0.380700	+0.380700
2	0.2	0.438628	+0.198705
3	0.4	0.496556	+0.019001
4	0.6	0.554484	−0.147325
5	0.8	0.612412	−0.274899
6	1.0	0.670340	−0.329660
7	1.2	0.728268	−0.289601
8	1.4	0.786196	−0.163858
9	1.6	0.844125	+0.013215

10	1.8	0.902053	+0.205171
11	2.0	0.959981	+0.388552

Мера отклонения и СКО:

$$S_1 = \sum_{i=1}^{11} \varepsilon_i^2 = 0.694687$$

$$\delta_1 = \sqrt{S_1/n} = \sqrt{0.694687/11} = 0.251$$

1.3 Квадратичная аппроксимация

В качестве эмпирической формулы берём полином второй степени $\varphi_2(x) = a_0 + a_1x + a_2x^2$. Сумма квадратов отклонений:

$$S(a_0, a_1, a_2) = \sum_{i=1}^n (a_0 + a_1x_i + a_2x_i^2 - y_i)^2 \rightarrow \min$$

Приравнивая нулю частные производные $\partial S/\partial a_k$ при $k = 0, 1, 2$, получаем систему нормальных уравнений:

$$\begin{cases} a_0 \cdot n + a_1 \cdot S_X + a_2 \cdot S_{XX} = S_Y \\ a_0 \cdot S_X + a_1 \cdot S_{XX} + a_2 \cdot S_{XXX} = S_{XY} \\ a_0 \cdot S_{XX} + a_1 \cdot S_{XXX} + a_2 \cdot S_{XXXX} = S_{XXY} \end{cases}$$

где дополнительно к ранее вычисленным:

$$S_{XXX} = \sum_{i=1}^n x_i^3, \quad S_{XXXX} = \sum_{i=1}^n x_i^4, \quad S_{XXY} = \sum_{i=1}^n x_i^2 y_i$$

По таблице 1:

$$S_{XXX} = 24.200000, \quad S_{XXXX} = 40.532800, \quad S_{XXY} = 11.905116$$

Подставляя суммы, получаем СЛАУ 3×3:

$$\begin{cases} 11.000000 \cdot a_0 + 11.000000 \cdot a_1 + 15.400000 \cdot a_2 = 7.373743 \\ 11.000000 \cdot a_0 + 15.400000 \cdot a_1 + 24.200000 \cdot a_2 = 8.648161 \\ 15.400000 \cdot a_0 + 24.200000 \cdot a_1 + 40.532800 \cdot a_2 = 11.905116 \end{cases}$$

$$a_0 = -0.041923, \quad a_1 = 1.698382, \quad a_2 = -0.704371$$

Итоговая модель: $\varphi_2(x) = -0.041923 + 1.698382x - 0.704371x^2$.

Таблица 3

Квадратичная аппроксимация: значения и отклонения

i	x_i	$\varphi_2(x_i)$	ε_i
1	0.0	-0.041923	-0.041923
2	0.2	0.269579	+0.029656
3	0.4	0.524731	+0.047176
4	0.6	0.723533	+0.021724
5	0.8	0.865986	-0.021326
6	1.0	0.952089	-0.047911
7	1.2	0.981842	-0.036027
8	1.4	0.955245	+0.005191
9	1.6	0.872299	+0.041390
10	1.8	0.733004	+0.036122
11	2.0	0.537358	-0.034070

Мера отклонения и СКО:

$$S_2 = 0.013588, \quad \delta_2 = \sqrt{S_2/11} = 0.0351$$

1.4 Сравнение и выбор наилучшего приближения

Таблица 4
Сравнительная сводка

Модель	S	δ	R^2	Качество
$\varphi_1 = ax + b$	0.694687	0.2513	0.347	Недостаточна
$\varphi_2 = a_0 + a_1x + a_2x^2$	0.013588	0.0351	0.987	Высокая

Среднеквадратичное отклонение квадратичной модели в 0.251/0.035 $\approx$ 7.2 раза меньше, чем у линейной; мера S — почти в 51 раз. Согласно методике лекции наилучшая аппроксимация — **квадратичная**:

$$\varphi(x) = -0.041923 + 1.698382x - 0.704371x^2$$

2 Программная реализация задачи

Согласно заданию, в программе реализованы следующие модели:

Модель	Назначение
Линейная	$\varphi(x) = ax + b$

Полиномиальная 2-й степени	$\varphi(x) = a_0 + a_1x + a_2x^2$
Полиномиальная 3-й степени	$\varphi(x) = a_0 + a_1x + a_2x^2 + a_3x^3$
Экспоненциальная	$\varphi(x) = a \cdot e^{bx}$ (требуется $y_i > 0$)
Логарифмическая	$\varphi(x) = a \cdot \ln(x) + b$ (требуется $x_i > 0$)
Степенная	$\varphi(x) = a \cdot x^b$ (требуется $x_i > 0$ и $y_i > 0$)
Коэффициент детерминации R^2	Сообщение о качестве для всех моделей
Коэффициент корреляции Пирсона r	Только для линейной модели
Выбор наилучшей по $\min \delta$	Итоговое сравнение всех моделей

2.1 Рабочие формулы и блок-схемы реализованных моделей

2.1.1 Линейная модель

Минимизируется

$$S(a, b) = \sum_{i=1}^n (ax_i + b - y_i)^2$$

В обозначениях S_X , S_{XX} , S_Y , S_{XY} нормальная система выглядит так, как было разобрано в § 1.2; коэффициенты a , b находятся по правилу Крамера. Дополнительно для линейной модели вычисляется коэффициент корреляции Пирсона:

$$r = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^n (x_i - \bar{x})^2 \cdot \sum_{i=1}^n (y_i - \bar{y})^2}}$$

Порог трактовки r принят как в лекции: $|r| < 0.3$ — связь слабая, 0.3...0.5 — умеренная, 0.5...0.7 — заметная, 0.7...0.9 — высокая, 0.9...0.99 — весьма высокая.

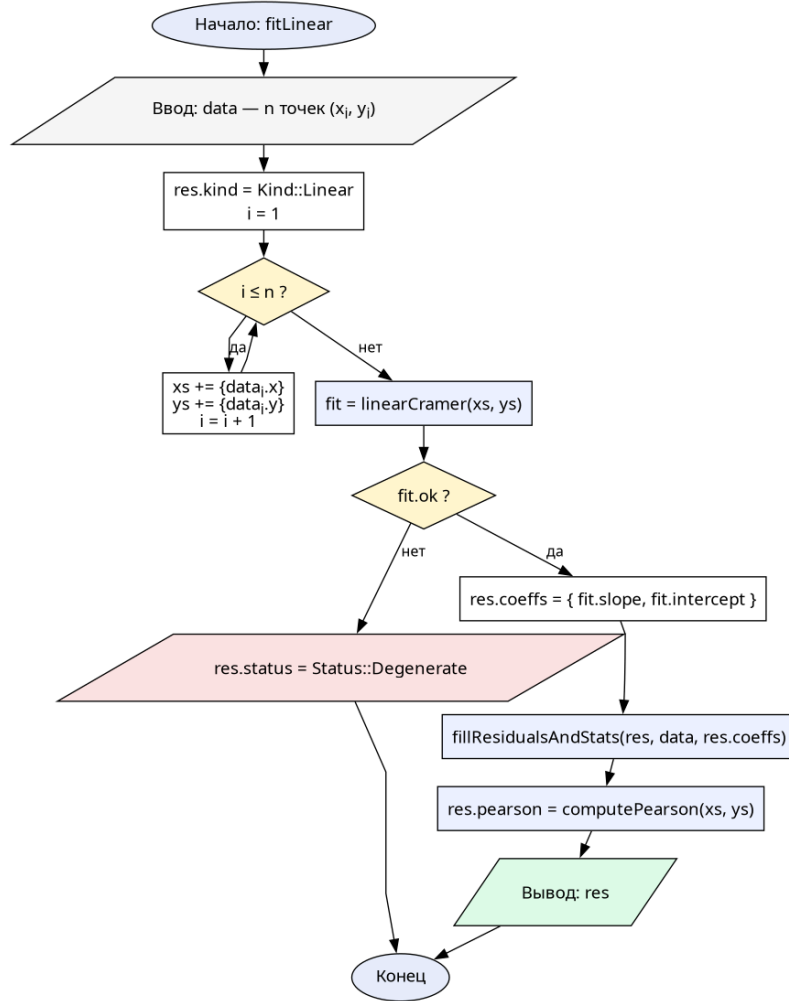


Рис. 1. Блок-схема линейной модели

2.1.2 Полиномиальные модели 2-й и 3-й степени

Для полинома степени m коэффициенты $a_0, \dots, a_m$ ищутся из системы $(m + 1) \times (m + 1)$:

$$\sum_{k=0}^m a_k \cdot S_{j+k} = T_j, \quad j = 0, \dots, m$$

где $S_p = \sum_i x_i^p$ — степенная сумма, $T_p = \sum_i x_i^p y_i$. Эта система симметрична и положительно определена для $n > m$ и попарно различных x_i . Решается тем же GaussSolver с выбором главного элемента, что и СЛАУ в ЛР №1, — что обеспечивает устойчивость даже при близких к вырожденным матрицам (например, на узких диапазонах x).

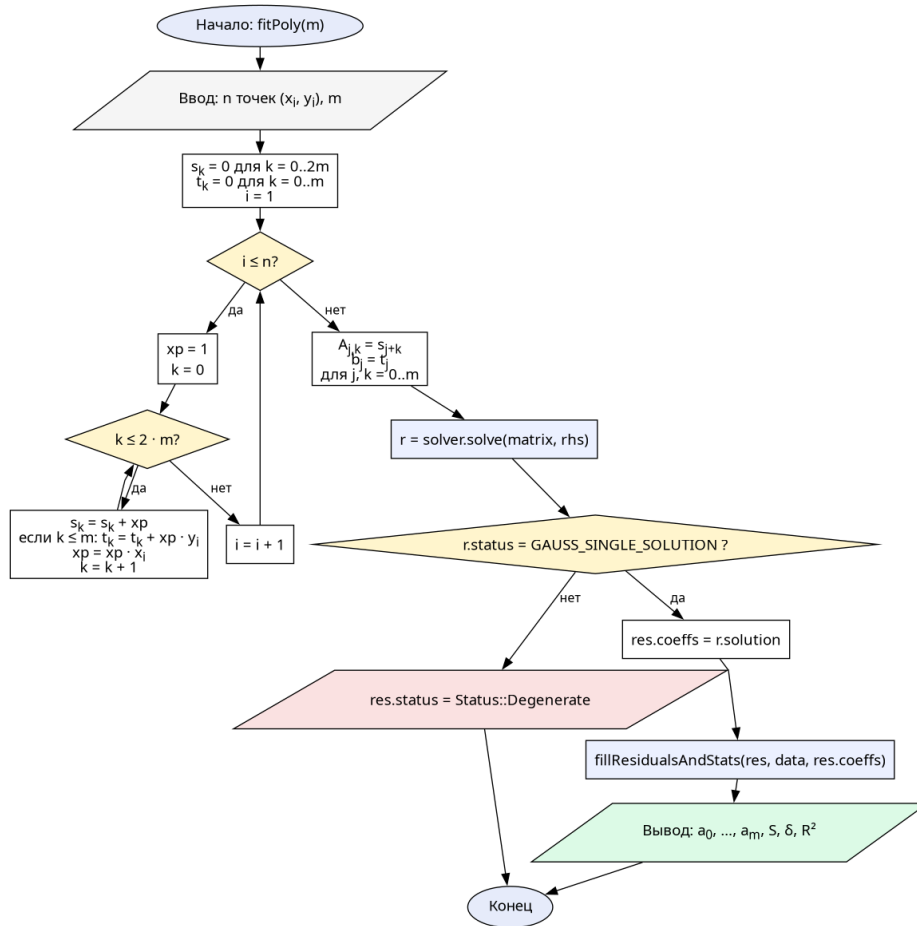


Рис. 2. Блок-схема общей полиномиальной модели степени m

2.1.3 Экспоненциальная модель

$\varphi(x) = a \cdot e^{bx}$. Линеаризация — переход к $Y = \ln y$:

$$\ln \varphi(x) = \ln a + bx \rightarrow Y = A + Bx, \quad A = \ln a, \quad B = b$$

В преобразованных координатах (x_i, Y_i) строится обычная линейная регрессия (правило Крамера). Из найденных A, B восстанавливаются исходные параметры $a = e^A, b = B$. Применимость требует $y_i > 0$ — иначе $\ln y_i$ не определён.

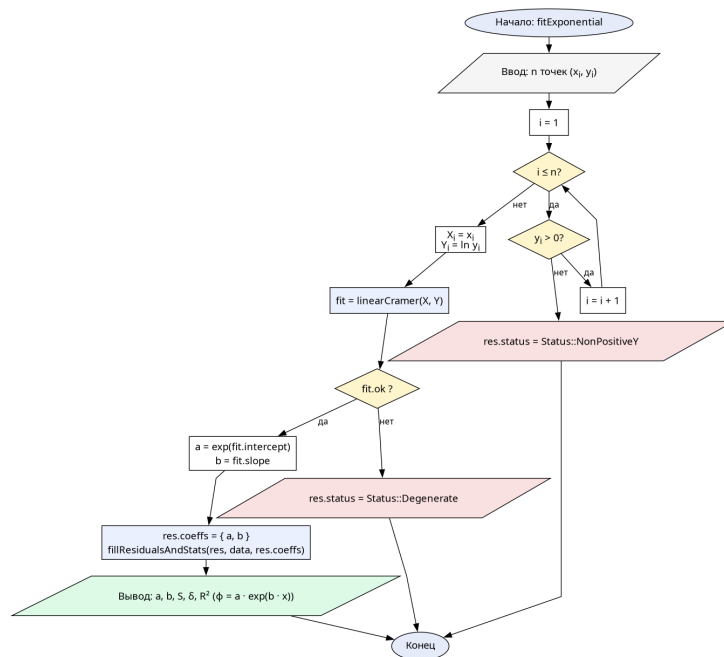


Рис. 3. Блок-схема экспоненциальной модели

2.1.4 Логарифмическая модель

$\varphi(x) = a \cdot \ln(x) + b$. Линеаризация – переход к $X = \ln x$:

$$\varphi(x) = aX + b \rightarrow y = aX + b$$

Применяется линейная регрессия по (X_i, y_i) . В отличие от экспоненциальной, для логарифмической обратная подстановка не нужна – коэффициенты a, b совпадают с найденными. Требуется $x_i > 0$.

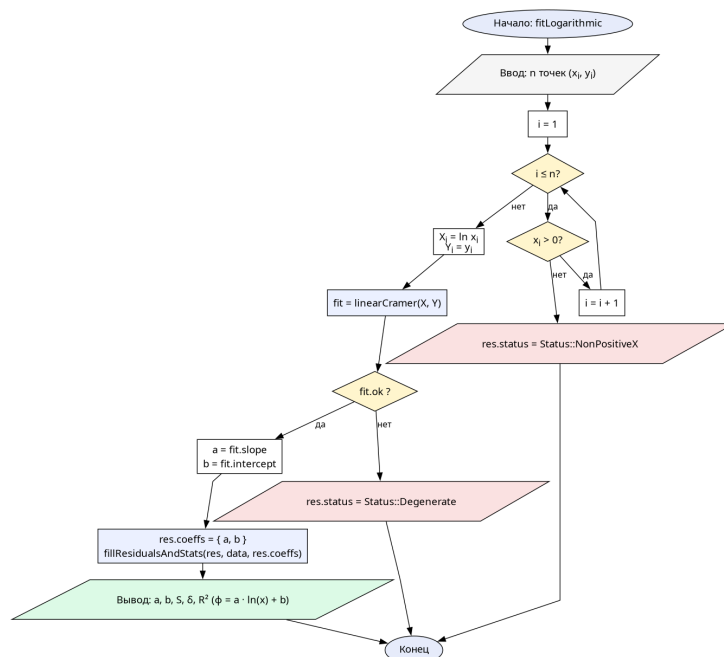


Рис. 4. Блок-схема логарифмической модели

2.1.5 Степенная модель

$\varphi(x) = a \cdot x^b$. Линеаризация — **двойная**, переход к $X = \ln x$, $Y = \ln y$:

$$\ln \varphi(x) = \ln a + b \ln x \rightarrow Y = A + BX, \quad A = \ln a, \quad B = b$$

После линейной регрессии по (X_i, Y_i) восстанавливаем $a = e^A$, $b = B$. Требуется одновременно $x_i > 0$ и $y_i > 0$ — это самая строгая по применимости модель из всех шести.

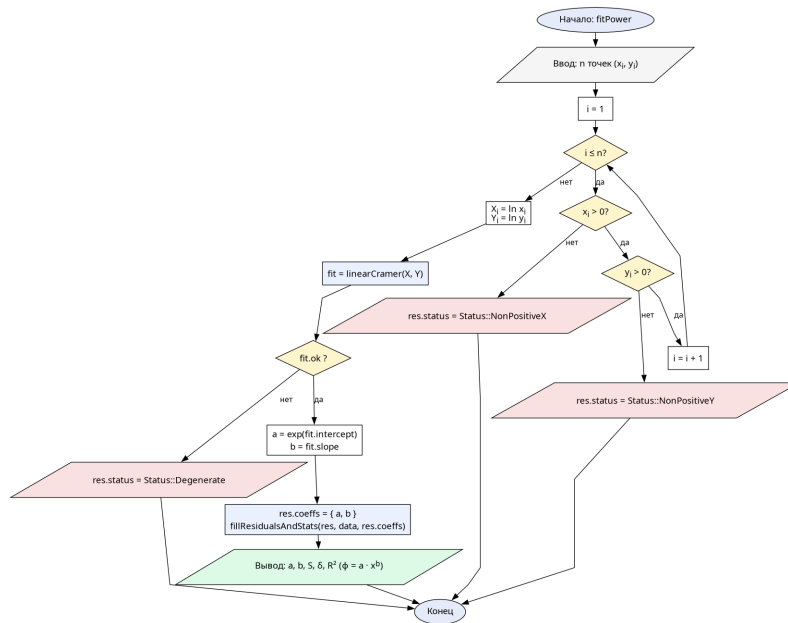


Рис. 5. Блок-схема степенной модели

2.2 Вспомогательные функции, общие для всех методов

Каждая из шести `fit*`-функций опирается на одну и ту же связку универсальных подпрограмм: `phiAt` подменяет конкретный вид модели единым диспетчером значения $\varphi(x)$; `linearCramer` решает нормальную систему 2×2 , к которой сводятся линейная и все три линеаризованные модели (экспоненциальная, логарифмическая, степенная); `fillResidualsAndStats` собирает массивы φ_i , ε_i и статистики S , δ , R^2 ; `computeR2` и `computePearson` — отдельно выделенные численные процедуры, на которые `fillResidualsAndStats` опирается. Это позволило вынести алгоритмические инварианты в одно место и в `fit*`-функциях оставить только построение нормальной системы / линеаризацию.

2.2.1 Диспетчер $\varphi(x)$ — phiAt

Получает дискриминатор `kind` и массив коэффициентов a ; возвращает значение модельной функции $\varphi(x)$. На него опираются `fillResidualsAndStats` (расчёт φ_i для всех 11 точек) и публичная `evaluate` (точка на интерполяционном графике). Реализация — серия `switch`-проверок:

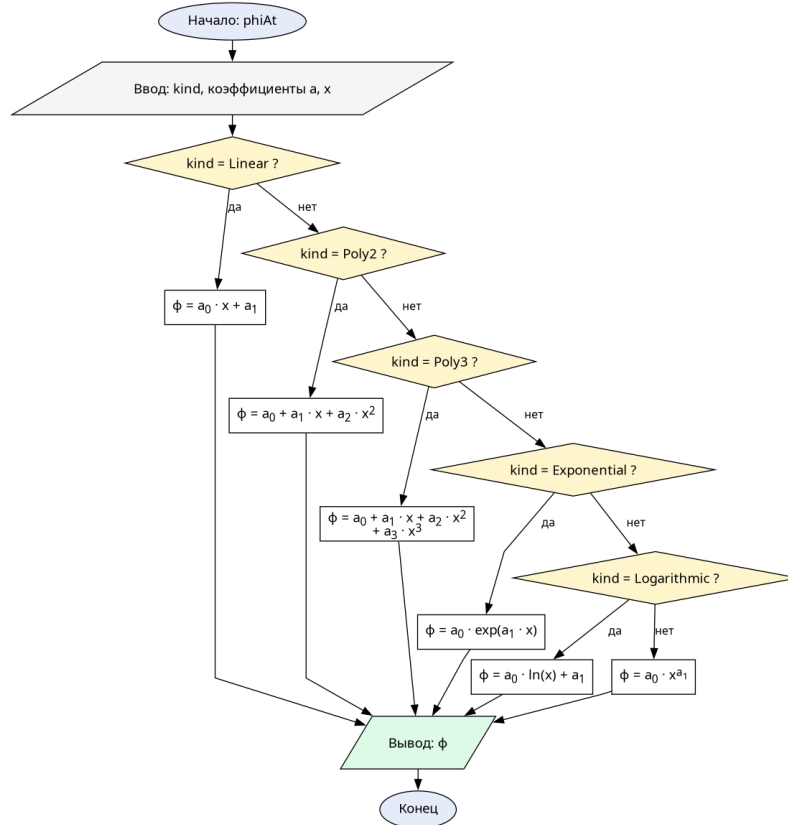


Рис. 6. Блок-схема `phiAt` — универсальный диспетчер $\varphi(x)$

2.2.2 Решение нормальной системы 2×2 — `linearCramer`

Используется в `fitLinear` напрямую, а также в `fitExponential`, `fitLogarithmic`, `fitPower` после линеаризации (логарифмирование одного или обоих координат). Накапливает S_X , S_{XX} , S_Y , S_{XY} за один проход, по правилу Крамера находит наклон и сдвиг прямой; при вырожденной матрице возвращает `ok = нет`.

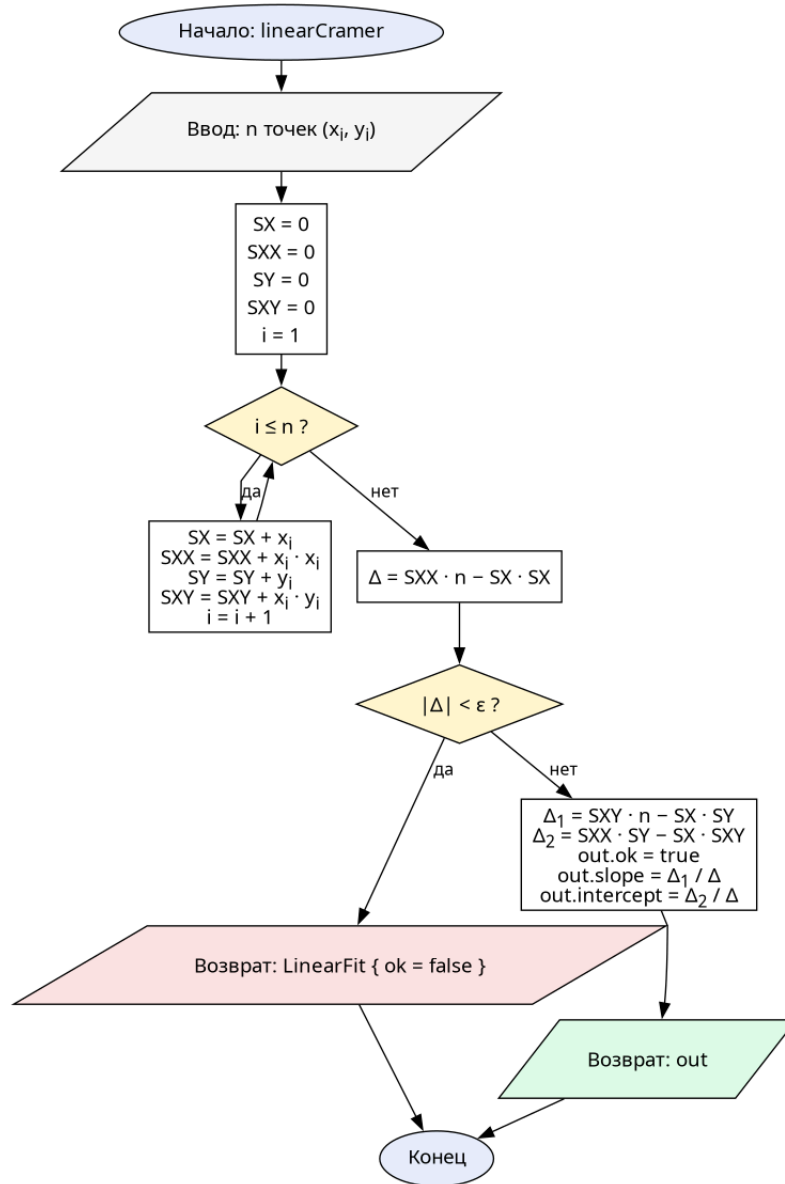


Рис. 7. Блок-схема `linearCramer` — общая 2×2 -регрессия

2.2.3 Коэффициент детерминации — `computeR2`

Универсальная мера качества: считает средний $\bar{y}$ за один проход, затем за второй — суммы квадратов остатков $SS_{\text{res}} = \sum (y_i - \varphi_i)^2$ и полной вариации $SS_{\text{tot}} = \sum (y_i - \bar{y})^2$. Возвращает $R^2 = 1 - SS_{\text{res}}/SS_{\text{tot}}$; при $SS_{\text{tot}} \rightarrow 0$ (плоский набор данных) защищается от деления почти на ноль: $R^2 = 1$, если $SS_{\text{res}} \rightarrow 0$, иначе $R^2 = 0$.

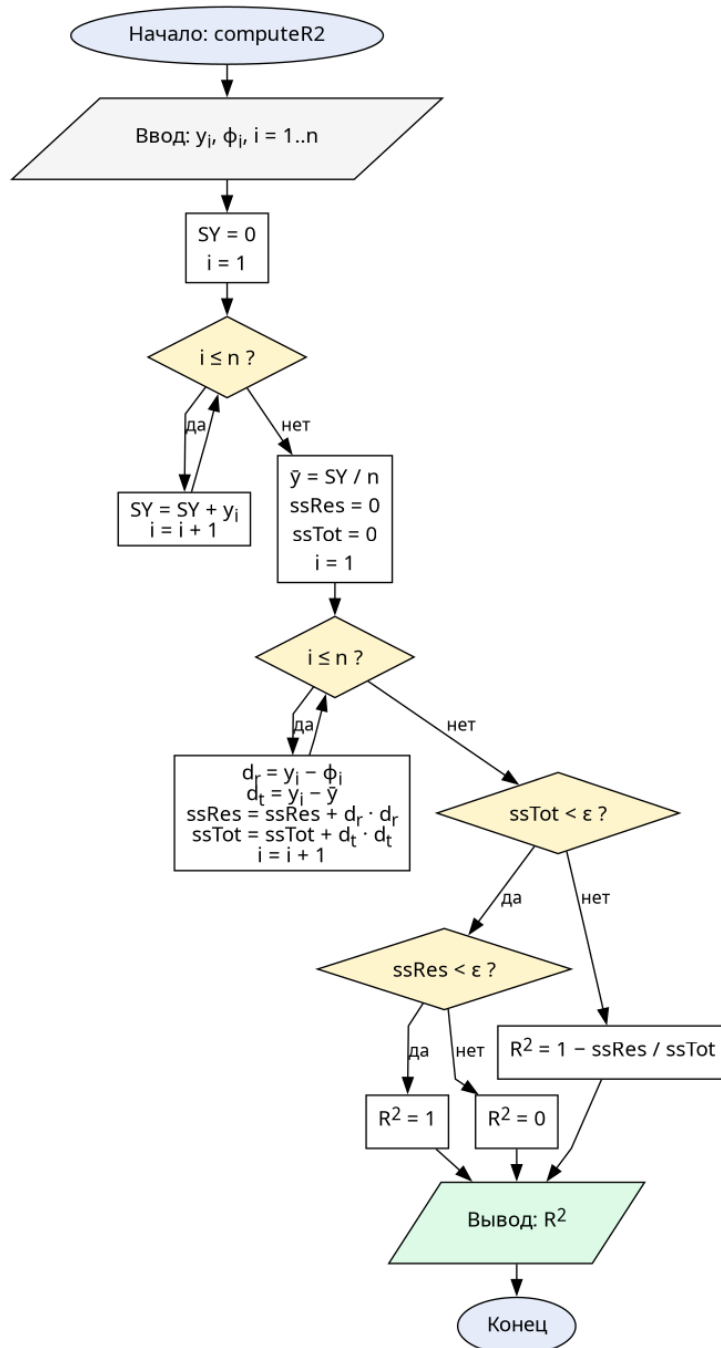


Рис. 8. Блок-схема `computeR2`

2.2.4 Коэффициент корреляции Пирсона — `computePearson`

Применяется только для линейной модели, но реализован как самостоятельная подпрограмма по той же логике двух проходов: средние $\bar{x}, \bar{y} \rightarrow$ числитель и знаменатели $r = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum (x_i - \bar{x})^2 \cdot \sum (y_i - \bar{y})^2}}$. При обнулённом знаменателе возвращает $r = 0$.

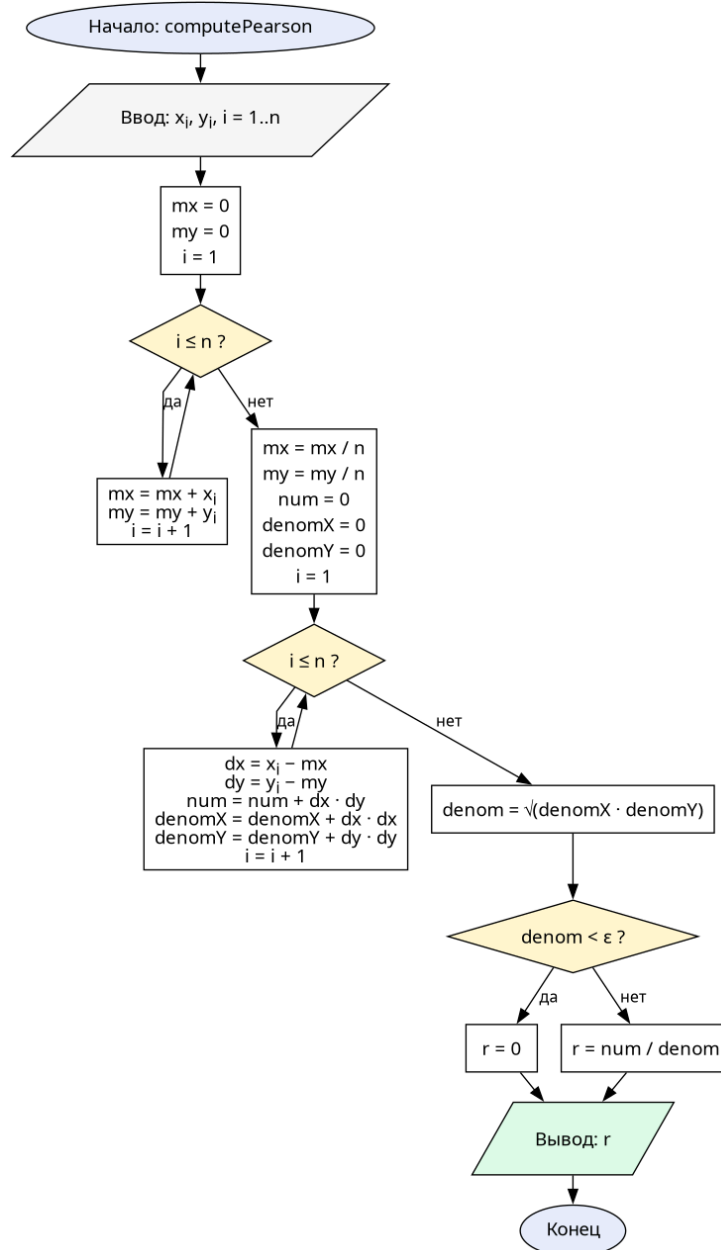


Рис. 9. Блок-схема `computePearson`

2.2.5 Общая постобработка — `fillResidualsAndStats`

Эта подпрограмма унифицирована для всех моделей: получает на вход коэффициенты и сами точки, считает φ_i через универсальный диспетчер `phiAt`, заполняет массив отклонений $\varepsilon_i = \varphi_i - y_i$, агрегирует:

$$S = \sum_{i=1}^n \varepsilon_i^2, \quad \delta = \sqrt{S/n}, \quad R^2 = 1 - S / \text{SST}$$

где $\text{SST} = \sum_i (y_i - \bar{y})^2$. Защита от деления на ноль вынесена в `computeR2`; все статистики вычисляются строго в исходных координатах.

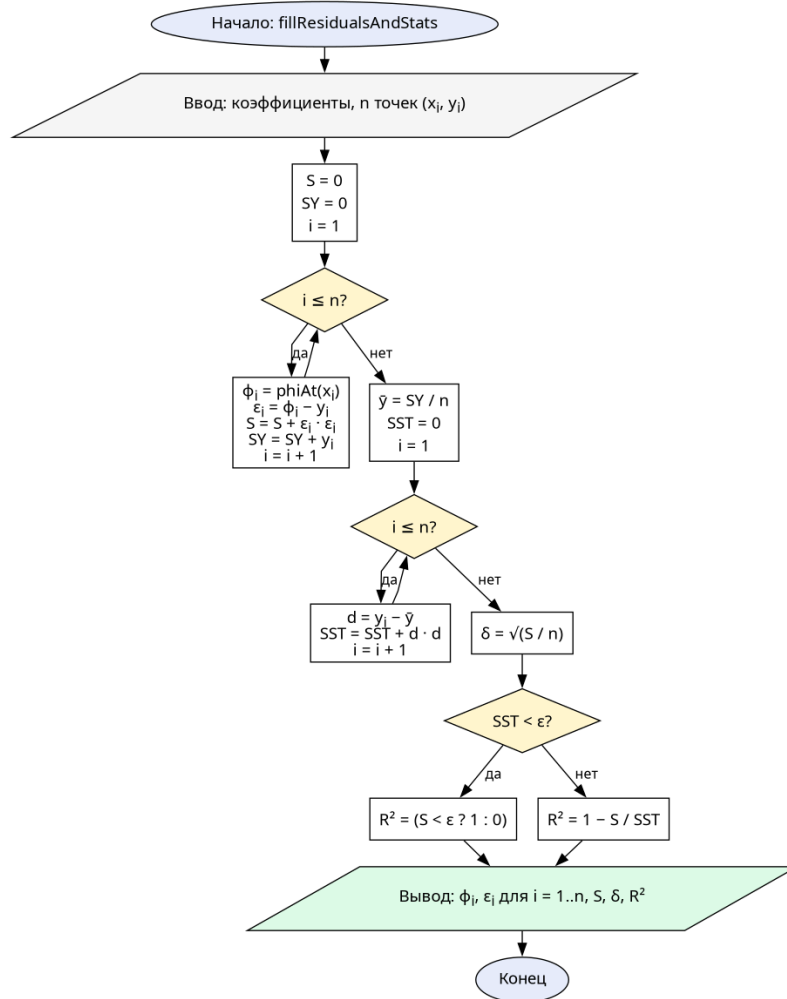


Рис. 10. Блок-схема `fillResidualsAndStats`

2.2.6 Выбор наилучшей модели — `pickBest`

После расчёта всех шести моделей выбирается та, у которой статус `Готово` и среднеквадратичное отклонение δ минимально. Модели, которые отказали (например, логарифмическая на $x_i \leq 0$), в выборе не участвуют.

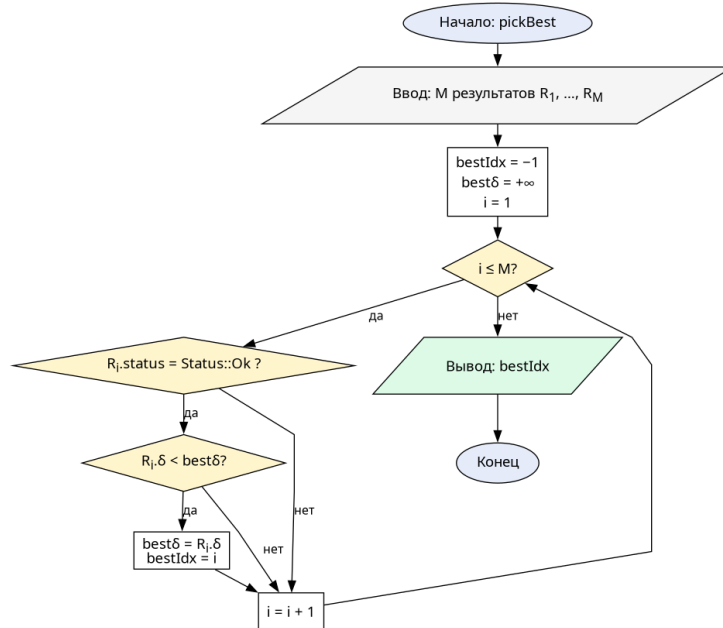


Рис. 11. Блок-схема pickBest

2.3 Сообщение по коэффициенту детерминации R^2

Для всех шести моделей в интерфейсе выводится текстовое сообщение по критерию лекции:

Диапазон R^2	Сообщение
$R^2 \geq 0.95$	Высокая точность аппроксимации
$0.75 \leq R^2 < 0.95$	Удовлетворительная аппроксимация
$0.50 \leq R^2 < 0.75$	Слабая аппроксимация
$R^2 < 0.50$	Точность недостаточна

2.4 Исходный код основных элементов расчёта

Заголовочный файл `calc/Approximation.hpp` — публичный API библиотеки:

```

#pragma once

#include <cstdlib>
#include <optional>
#include <vector>

namespace approx {

enum class Kind {
    Linear,
    Poly2,
    Poly3,
    Exponential,

```

```

    Logarithmic,
    Power
};

enum class Status {
    Ok,
    TooFew,
    TooMany,
    NonPositiveX,
    NonPositiveY,
    Degenerate
};

struct Point {
    double x = 0.0;
    double y = 0.0;
};

struct Result {
    Kind kind = Kind::Linear;
    Status status = Status::Ok;
    std::vector<double> coeffs;
    std::vector<double> phi;
    std::vector<double> eps;
    double S = 0.0;
    double delta = 0.0;
    double r2 = 0.0;
    std::optional<double> pearson;
};

class LeastSquaresApproximator {
public:
    static constexpr std::size_t kMinPoints = 4;
    static constexpr std::size_t kMaxPoints = 12;

    Result approximate(Kind kind, const std::vector<Point> &data) const;
    std::vector<Result> approximateAll(const std::vector<Point> &data) const;

    static double evaluate(const Result &r, double x);
};

} // namespace approx

```

Реализация (calc/approximation.cpp) — все шесть fit*-функций, общий

fillResidualsAndStats, линеаризации:

```

#include "Approximation.hpp"

#include "Gauss.hpp"

#include <cmath>

namespace approx {

namespace {

constexpr double kEps = 1e-12;

double phiAt(Kind kind, const std::vector<double> &a, double x) {
    switch (kind) {

```

```

    case Kind::Linear:
        return a[0] * x + a[1];
    case Kind::Poly2:
        return a[0] + a[1] * x + a[2] * x * x;
    case Kind::Poly3:
        return a[0] + a[1] * x + a[2] * x * x + a[3] * x * x * x;
    case Kind::Exponential:
        return a[0] * std::exp(a[1] * x);
    case Kind::Logarithmic:
        return a[0] * std::log(x) + a[1];
    case Kind::Power:
        return a[0] * std::pow(x, a[1]);
    }
    return 0.0;
}

double computeR2(const std::vector<double> &y, const std::vector<double> &phi) {
    if (y.size() != phi.size() || y.empty()) {
        return 0.0;
    }
    double meanY = 0.0;
    for (double v : y) {
        meanY += v;
    }
    meanY /= static_cast<double>(y.size());

    double ssRes = 0.0;
    double ssTot = 0.0;
    for (std::size_t i = 0; i < y.size(); ++i) {
        ssRes += (y[i] - phi[i]) * (y[i] - phi[i]);
        ssTot += (y[i] - meanY) * (y[i] - meanY);
    }
    if (ssTot < kEps) {
        return ssRes < kEps ? 1.0 : 0.0;
    }
    return 1.0 - ssRes / ssTot;
}

double computePearson(const std::vector<double> &x,
                     const std::vector<double> &y) {
    if (x.size() != y.size() || x.empty()) {
        return 0.0;
    }
    double mx = 0.0;
    double my = 0.0;
    for (double v : x) {
        mx += v;
    }
    for (double v : y) {
        my += v;
    }
    mx /= static_cast<double>(x.size());
    my /= static_cast<double>(y.size());

    double num = 0.0;
    double denomX = 0.0;
    double denomY = 0.0;
    for (std::size_t i = 0; i < x.size(); ++i) {
        const double dx = x[i] - mx;
        const double dy = y[i] - my;

```

```

    num += dx * dy;
    denomX += dx * dx;
    denomY += dy * dy;
}
const double denom = std::sqrt(denomX * denomY);
if (denom < kEps) {
    return 0.0;
}
return num / denom;
}

void fillResidualsAndStats(Result &res, const std::vector<Point> &data,
                           const std::vector<double> &coeffs) {
    const std::size_t n = data.size();
    res.phi.assign(n, 0.0);
    res.eps.assign(n, 0.0);
    std::vector<double> ys(n);
    for (std::size_t i = 0; i < n; ++i) {
        res.phi[i] = phiAt(res.kind, coeffs, data[i].x);
        res.eps[i] = res.phi[i] - data[i].y;
        ys[i] = data[i].y;
    }

    double S = 0.0;
    for (double e : res.eps) {
        S += e * e;
    }
    res.S = S;
    res.delta = std::sqrt(S / static_cast<double>(n));
    res.r2 = computeR2(ys, res.phi);
}

struct LinearFit {
    bool ok = false;
    double slope = 0.0;
    double intercept = 0.0;
};

LinearFit linearCramer(const std::vector<double> &xs,
                      const std::vector<double> &ys) {
    LinearFit out;
    const std::size_t n = xs.size();
    if (n == 0 || n != ys.size()) {
        return out;
    }

    double SX = 0.0;
    double SXX = 0.0;
    double SY = 0.0;
    double SXY = 0.0;
    for (std::size_t i = 0; i < n; ++i) {
        SX += xs[i];
        SXX += xs[i] * xs[i];
        SY += ys[i];
        SXY += xs[i] * ys[i];
    }

    const double nD = static_cast<double>(n);
    const double det = SXX * nD - SX * SX;
    if (std::abs(det) < kEps) {

```

```

    return out;
}
const double d1 = SXY * nD - SX * SY;
const double d2 = SXX * SY - SX * SXY;
out.ok = true;
out.slope = d1 / det;
out.intercept = d2 / det;
return out;
}

Result fitLinear(const std::vector<Point> &data) {
    Result res;
    res.kind = Kind::Linear;

    std::vector<double> xs;
    std::vector<double> ys;
    xs.reserve(data.size());
    ys.reserve(data.size());
    for (const auto &p : data) {
        xs.push_back(p.x);
        ys.push_back(p.y);
    }

    const LinearFit fit = linearCramer(xs, ys);
    if (!fit.ok) {
        res.status = Status::Degenerate;
        return res;
    }

    res.coeffs = {fit.slope, fit.intercept};
    fillResidualsAndStats(res, data, res.coeffs);
    res.pearson = computePearson(xs, ys);
    return res;
}

Result fitPolyKind(Kind kind, int degree, const std::vector<Point> &data) {
    Result res;
    res.kind = kind;

    const int m = degree + 1;
    std::vector<double> sumPow(2 * degree + 1, 0.0);
    std::vector<double> sumXky(m, 0.0);

    for (const auto &p : data) {
        double xp = 1.0;
        for (int k = 0; k ≤ 2 * degree; ++k) {
            sumPow[k] += xp;
            if (k < m) {
                sumXky[k] += xp * p.y;
            }
            xp *= p.x;
        }
    }

    std::vector<std::vector<double>> matrix(m, std::vector<double>(m, 0.0));
    std::vector<double> rhs(m, 0.0);
    for (int j = 0; j < m; ++j) {
        for (int k = 0; k < m; ++k) {
            matrix[j][k] = sumPow[j + k];
        }
    }
}

```

```

    rhs[j] = sumXky[j];
}

GaussSolver solver;
const GaussResult r = solver.solve(matrix, rhs);
if (r.status != GAUSS_SINGLE_SOLUTION) {
    res.status = Status::Degenerate;
    return res;
}

res.coeffs = r.solution;
fillResidualsAndStats(res, data, res.coeffs);
return res;
}

Result fitExponential(const std::vector<Point> &data) {
    Result res;
    res.kind = Kind::Exponential;

    for (const auto &p : data) {
        if (p.y ≤ kEps) {
            res.status = Status::NonPositiveY;
            return res;
        }
    }

    std::vector<double> xs;
    std::vector<double> ys;
    xs.reserve(data.size());
    ys.reserve(data.size());
    for (const auto &p : data) {
        xs.push_back(p.x);
        ys.push_back(std::log(p.y));
    }

    const LinearFit fit = linearCramer(xs, ys);
    if (!fit.ok) {
        res.status = Status::Degenerate;
        return res;
    }

    res.coeffs = {std::exp(fit.intercept), fit.slope};
    fillResidualsAndStats(res, data, res.coeffs);
    return res;
}

Result fitLogarithmic(const std::vector<Point> &data) {
    Result res;
    res.kind = Kind::Logarithmic;

    for (const auto &p : data) {
        if (p.x ≤ kEps) {
            res.status = Status::NonPositiveX;
            return res;
        }
    }

    std::vector<double> xs;
    std::vector<double> ys;
    xs.reserve(data.size());

```

```

ys.reserve(data.size());
for (const auto &p : data) {
    xs.push_back(std::log(p.x));
    ys.push_back(p.y);
}

const LinearFit fit = linearCramer(xs, ys);
if (!fit.ok) {
    res.status = Status::Degenerate;
    return res;
}

res.coeffs = {fit.slope, fit.intercept};
fillResidualsAndStats(res, data, res.coeffs);
return res;
}

Result fitPower(const std::vector<Point> &data) {
    Result res;
    res.kind = Kind::Power;

    for (const auto &p : data) {
        if (p.x ≤ kEps) {
            res.status = Status::NonPositiveX;
            return res;
        }
        if (p.y ≤ kEps) {
            res.status = Status::NonPositiveY;
            return res;
        }
    }

    std::vector<double> xs;
    std::vector<double> ys;
    xs.reserve(data.size());
    ys.reserve(data.size());
    for (const auto &p : data) {
        xs.push_back(std::log(p.x));
        ys.push_back(std::log(p.y));
    }

    const LinearFit fit = linearCramer(xs, ys);
    if (!fit.ok) {
        res.status = Status::Degenerate;
        return res;
    }

    res.coeffs = {std::exp(fit.intercept), fit.slope};
    fillResidualsAndStats(res, data, res.coeffs);
    return res;
}

} // namespace

Result
LeastSquaresApproximator::approximate(Kind kind,
                                       const std::vector<Point> &data) const {
    Result res;
    res.kind = kind;

```

```

    if (data.size() < kMinPoints) {
        res.status = Status::TooFew;
        return res;
    }
    if (data.size() > kMaxPoints) {
        res.status = Status::TooMany;
        return res;
    }

    switch (kind) {
    case Kind::Linear:
        return fitLinear(data);
    case Kind::Poly2:
        return fitPolyKind(Kind::Poly2, 2, data);
    case Kind::Poly3:
        return fitPolyKind(Kind::Poly3, 3, data);
    case Kind::Exponential:
        return fitExponential(data);
    case Kind::Logarithmic:
        return fitLogarithmic(data);
    case Kind::Power:
        return fitPower(data);
    }
    return res;
}

std::vector<Result> LeastSquaresApproximator::approximateAll(
    const std::vector<Point> &data) const {
    return {
        approximate(Kind::Linear, data),
        approximate(Kind::Poly2, data),
        approximate(Kind::Poly3, data),
        approximate(Kind::Exponential, data),
        approximate(Kind::Logarithmic, data),
        approximate(Kind::Power, data),
    };
}

double LeastSquaresApproximator::evaluate(const Result &r, double x) {
    if (r.status != Status::Ok) {
        return std::nan("");
    }
    if ((r.kind == Kind::Logarithmic || r.kind == Kind::Power) && x ≤ kEps) {
        return std::nan("");
    }
    const double y = phiAt(r.kind, r.coeffs, x);
    return std::isfinite(y) ? y : std::nan("");
}

} // namespace approx

```


2.5 Интерфейс приложения

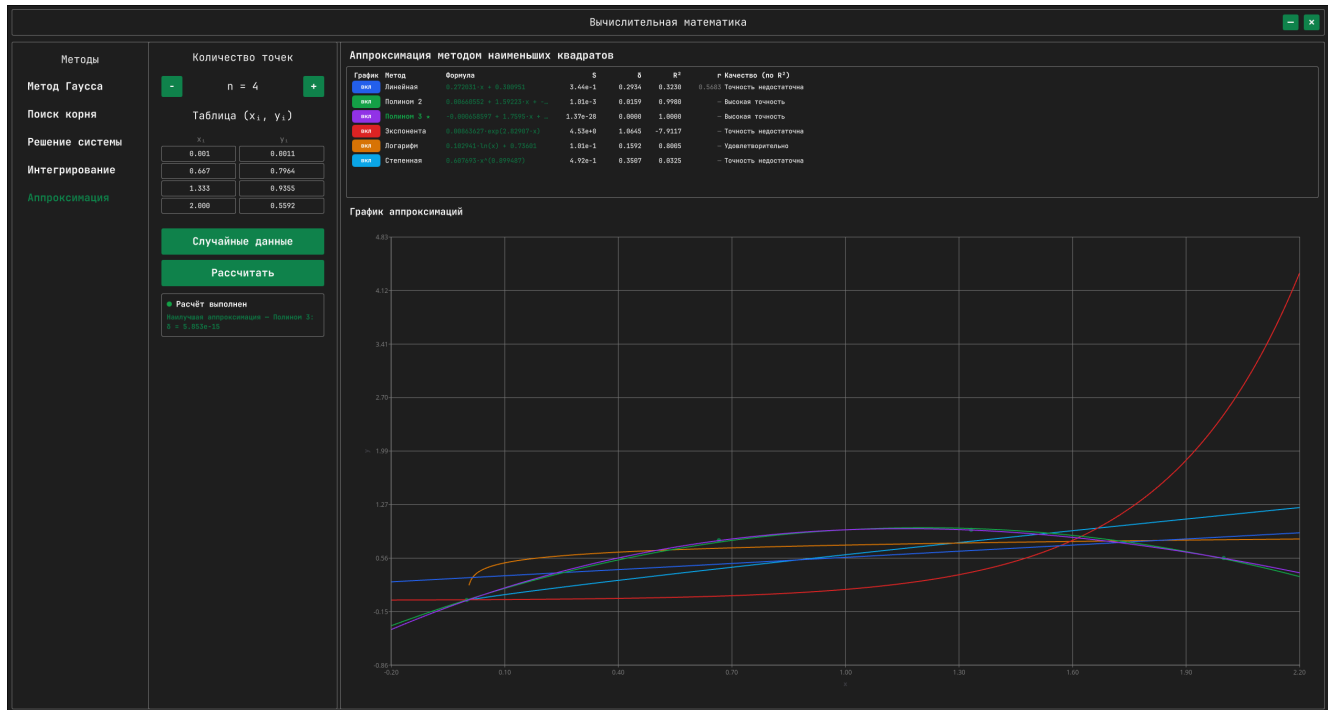


Рис. 12. Внешний вид вкладки «Аппроксимация» в приложении Полный исходный код проекта (Qt6/QML/C++ десктоп-приложение) доступен в репозитории:

git.oriptal.dev/cadmin/calc-math-lab2

3 Выводы

В ходе лабораторной работы изучен и программно реализован метод наименьших квадратов для шести моделей: линейной, полиномиальных 2-й и 3-й степени, экспоненциальной, логарифмической и степенной. Для всех шести модулей программа выводит коэффициенты, массивы φ_i и ε_i , меру S , СКО δ и коэффициент детерминации R^2 с пороговым сообщением; для линейной — дополнительно коэффициент корреляции Пирсона r . Линеаризованные модели корректно отказывают на некорректных данных: логарифмическая и степенная требуют $x_i > 0$, экспоненциальная и степенная — $y_i > 0$; в этих случаях соответствующая строка в таблице помечается оранжевой надписью «Неприменимо: требуется ...» и в выборе наилучшего варианта не участвует.

Для варианта №5 ($y = 6x/(x^4 + 5)$, $x \in [0, 2]$, $h = 0.2$) в вычислительной части построены линейная ($\delta_1 = 0.251$) и квадратичная

($\delta_2 = 0.0351$) аппроксимации по 11 точкам; квадратичная модель в 7 раз точнее линейной по δ и в 51 раз — по мере S , что согласуется с присутствием выраженного максимума у исходной функции и неспособностью прямой такой максимум описать. Коэффициент детерминации $R^2 = 0.347$ у линейной попадает в категорию «точность недостаточна», а $R^2 = 0.987$ у квадратичной — в категорию «высокая точность аппроксимации». Итоговый выбор — квадратичная модель.

Стоит явно отметить корректное ограничение области применимости. Согласно п. 15 контрольных вопросов лекции и пункту 7 задания, графики строятся «полностью на всём исследуемом интервале с запасом» в $\pm 10\%$ ширины, но **экстраполяция** за пределы $[x_{\min}, x_{\max}]$ исходной таблицы не считается математически обоснованной — приближение методом наименьших квадратов гарантирует малое СКО только внутри области выборки и за её пределами может разойтись с исходной функцией сколь угодно сильно (например, степенная и экспоненциальная модели монотонно растут, тогда как функция варианта №5 при $x \rightarrow \infty$ стремится к нулю). Поэтому полученные коэффициенты следует трактовать как эмпирическое описание поведения функции на $[0, 2]$, а не как её аналитический закон.